

45 Days Summer Training Program

Assignment 4:

1. Write a program that takes a list of numbers and prints all prime numbers in the list using loops. Do not use any built-in prime-checking functions.

```
➤ numbers = [2, 4, 5, 7, 9, 11, 15, 17,  
              20, 23]
```

```
    print("Prime numbers in the list  
are:")
```

```
for num in numbers:
```

```
    if num > 1:
```

```
        is_prime = True
```

```
        for i in range(2, num):
```

```
            if num % i == 0:
```

```
                is_prime = False
```

```
                break
```

```
        if is_prime:
```

```
            print(num)
```

- output :-

Prime numbers in the list are:

2

5

7

11

17

23

2. Write a program that generates a multiplication table (1–10) In matrix format using nested loops.

```
➤ for i in range(1, 11):
    for j in range(1, 11):
        print(i * j, end="\t")
    print()
```

- output:-

1 2 3 4 5 6 7 8 9 10

2 4 6 8 10 12 14 16 18 20

3 6 9 12 15 18 21 24 27 30

...

10 20 30 40 50 60 70 80 90 100

3. Write a user-defined function that accepts a list of integers and returns:

- Count of even numbers
- Sum of odd numbers
- Use loops and conditional statements inside the function.

```
➤ def count_even_sum_odd(numbers):  
    even_count = 0  
    odd_sum = 0
```

```
    for num in numbers:  
        if num % 2 == 0:  
            even_count += 1  
        else:  
            odd_sum += num
```

```
    return even_count, odd_sum
```

```
numbers = [1, 2, 3, 4, 5, 6, 7]  
even_count, odd_sum =  
count_even_sum_odd(numbers)
```

```
print("Count of even numbers:",  
      even_count)
```

```
print("Sum of odd numbers:",  
odd_sum)
```

- output :-

Count of even numbers: 3

Sum of odd numbers: 16

4. Create a function that calculates simple interest.
Use default arguments for rate and time.
Demonstrate calling the function using both positional
and keyword arguments.

```
➤ def simple_interest(principal, rate=5,  
time=2):  
    si = (principal * rate * time) / 100  
    return si
```

```
print("Simple Interest:",  
simple_interest(1000))
```

```
print("Simple Interest:",  
simple_interest(1000, 8, 3))
```

```
print("Simple Interest:",  
      simple_interest(principal=1000,  
                      rate=10, time=4))
```

- output :-

Simple Interest: 100.0

Simple Interest: 240.0

Simple Interest: 400.0

5. Create a Student class with attributes: name, marks. Add a method to calculate grade based on marks.

➤ class Student:

```
def __init__(self, name, marks):  
    self.name = name  
    self.marks = marks
```

```
def calculate_grade(self):  
    if self.marks >= 90:  
        return "A"  
    elif self.marks >= 75:  
        return "B"  
    elif self.marks >= 50:  
        return "C"  
    else:
```

```
return "F"
```

```
s1 = Student("Pankaj", 85)
```

```
print("Name:", s1.name)
```

```
print("Marks:", s1.marks)
```

```
print("Grade:", s1.calculate_grade())
```

- output :-

Name: Pankaj

Marks: 85

Grade: B

6. Create at least two objects and display their grades.

➤ class Student:

```
def __init__(self, name, marks):
```

```
    self.name = name
```

```
    self.marks = marks
```

```
def calculate_grade(self):
```

```
    if self.marks >= 90:
```

```
        return "A"
```

```
    elif self.marks >= 75:
```

```
        return "B"
    elif self.marks >= 50:
        return "C"
    else:
        return "F"
```

```
s1 = Student("Pankaj", 85)
s2 = Student("Rahul", 92)
```

```
print(s1.name, "-",
      s1.calculate_grade())
print(s2.name, "-",
      s2.calculate_grade())
```

- output :-

Pankaj - B
Rahul – A

7. Create a BankAccount class with private attributes for balance. Provide methods to deposit and withdraw money. Ensure balance cannot be accessed directly.

```
➤ class BankAccount:
    def __init__(self, balance):
```

```
self.__balance = balance
```

```
def deposit(self, amount):  
    self.__balance += amount
```

```
def withdraw(self, amount):  
    if amount <= self.__balance:  
        self.__balance -= amount  
    else:  
        print("Insufficient balance")
```

```
def get_balance(self):  
    return self.__balance
```

```
account = BankAccount(5000)
```

```
account.deposit(1000)  
account.withdraw(2000)
```

```
print("Balance:",  
      account.get_balance())
```

- output :-

Balance: 4000

8. Write a program that takes two numbers from the user and performs division.

- Handle the following exceptions:
- Division by zero
- Invalid input type

```
try:
    num1 = int(input("Enter first
number: "))
    num2 = int(input("Enter second
number: "))

    result = num1 / num2
    print("Result:", result)

except ZeroDivisionError:
    print("Error: Division by zero is not
allowed.")

except ValueError:
    print("Error: Invalid input. Please
enter numbers only.")
```

- Output (Division by Zero):

Enter first number: 10

Enter second number:

0

Error: Division by zero

is not allowed.

- Output (Invalid Input):

Enter first number: 10

Enter second number: abc

Error: Invalid input. Please

enter numbers only.

9. Write a program that:

- Creates a file
- Writes student data (name and marks)
- Reads and displays the content

```
➤ file = open("student.txt", "w")  
file.write("Name: Pankaj\n")  
file.write("Marks: 85\n")  
file.close()
```

```
file = open("student.txt", "r")  
data = file.read()  
print(data)  
file.close()
```

- output :-

Name: Pankaj

Marks: 85

10. Write a program that reads numbers from a file and calculates:

- Total
- Average
- Handle the case if the file does not exist.

➤ try:

```
file = open("numbers.txt", "r")
```

```
numbers = file.readlines()
```

```
total = 0
```

```
count = 0
```

```
for num in numbers:
```

```
    total += int(num)
```

```
    count += 1
```

```
average = total / count
```

```
print("Total:", total)
```

```
print("Average:", average)
```

```
file.close()
```

```
except FileNotFoundError:  
    print("Error: File does not exist.")
```

- Example :-

10

20

30

40

50

- Output:-

Total: 150

Average: 30.0

Tfg